

Indicaciones específicas:

- Esta evaluación contiene 7 páginas.
 - La solución de la tarea deber ser entregada con un link al repositorio en GitHub.
-

Tarea #1: Tensor++

Desarrollo de una librería de tensores en C++

1 Descripción general

En esta tarea se deberá diseñar e implementar una librería en C++ llamada **Tensor++**, inspirada en bibliotecas científicas como *NumPy* y *PyTorch*. El objetivo es crear una clase **Tensor** capaz de manejar **tensores de hasta 3 dimensiones**, realizar operaciones matemáticas avanzadas y ser utilizada en una aplicación realista de procesamiento de datos.

La tarea debe realizarse en grupos de **dos personas**.

2 Restricciones generales

- Los tensores deben tener un máximo de **3 dimensiones** (1D, 2D o 3D).
- Los datos del tensor deben almacenarse utilizando **memoria dinámica**, implementada por el alumno mediante **una sola** de las siguientes opciones:
 - Array dinámico contiguo (`double*`)
 - Matriz dinámica (`double**`)
 - Matriz tridimensional dinámica (`double***`)
- No se permite el uso de librerías externas para manejo de tensores.

3 Clase Tensor

3.1 Constructor principal

```
Tensor(const std::vector<size_t>& shape ,  
        const std::vector<double>& values);
```

- **shape**: dimensiones del tensor (máximo 3 valores).
- **values**: valores almacenados en orden contiguo.
- El tamaño de **values** debe coincidir con el producto de las dimensiones.

Las listas entre llaves `{}` utilizadas en los ejemplos se reciben como `std::vector` mediante *initializer lists*.

3.2 Creación de tensores pre-definidos

La clase `Tensor` debe proveer métodos estáticos para facilitar la creación de tensores, imitando el estilo de bibliotecas como *NumPy*. Estos métodos abstraen la gestión de memoria y permiten crear tensores de forma clara y segura.

```
Tensor A = Tensor::zeros({2, 3});
Tensor B = Tensor::ones({3, 3});
Tensor C = Tensor::random({2, 2}, 0.0, 1.0);
Tensor D = Tensor::arange(0, 6);
```

Uso esperado:

- **zeros:** crea un tensor con todas sus posiciones inicializadas en cero.
- **ones:** crea un tensor con todas sus posiciones inicializadas en uno.
- **random:** crea un tensor con valores aleatorios distribuidos uniformemente en el rango $[min, max)$.
- **arange:** crea un tensor unidimensional con valores secuenciales desde **start** hasta **end** (no inclusivo).

4 Gestión de Memoria y Ciclo de Vida

- **Constructor de Copia:** Debe realizar una copia profunda (*deep copy*) de los datos.
- **Asignador de Copia (operator=):** Debe liberar la memoria actual antes de copiar los nuevos datos, manejando la auto-asignación.
- **Constructor de Movimiento (move constructor):** Debe transferir la propiedad del puntero de datos del objeto origen al nuevo objeto, dejando al origen en un estado válido pero nulo.
- **Asignador de Movimiento (move operator=):** Debe liberar los recursos actuales y tomar posesión de los del objeto temporal.
- **Destructor:** Debe liberar toda la memoria dinámica reservada.

```
// Prototipos requeridos en la clase Tensor
Tensor(const Tensor& other);           // Copia
Tensor(Tensor&& other) noexcept;       // Movimiento
Tensor& operator=(const Tensor& other); // Asignacion copia
Tensor& operator=(Tensor&& other) noexcept; // Asignacion movimiento
~Tensor();                             // Destructor
```

%endlstlisting

5 Sobrecarga de operadores

Se deben implementar los siguientes operadores:

```
Tensor C = A + B;  
Tensor D = A - B;  
Tensor E = A * B;  
Tensor F = A * 2.0;
```

- Las dimensiones deben ser compatibles.
- En caso contrario, se debe lanzar una excepción.
- Los operandos no deben modificarse.
- Los operadores deben devolver nuevos tensores.

6 Modificación de dimensiones

6.1 view

El método `view` permite reinterpretar la forma de un tensor sin copiar los datos subyacentes (trasladar la información al tensor resultante), siempre que el número total de elementos se mantenga constante.

```
Tensor A = Tensor::arange(0, 12);  
Tensor B = A.view({3, 4});
```

Uso esperado:

- Cambiar la organización lógica de los datos.
- Optimizar el rendimiento evitando copias innecesarias de memoria.

El alumno debe garantizar que:

- No se deben copiar los datos (**move**).
- El tensor original permanezca válido.
- El número de dimensiones no exceda tres.

6.2 unsqueeze

El método `unsqueeze` permite agregar una dimensión de tamaño 1 en una posición específica del tensor, sin modificar los datos subyacentes.

```
Tensor A = Tensor::arange(0, 3);  
Tensor B = A.unsqueeze(0); % shape: {1,3}  
Tensor C = A.unsqueeze(1); % shape: {3,1}
```

Uso esperado:

- Preparar tensores para operaciones matriciales.
- Adaptar dimensiones para concatenación o suma con bias.
- Simular el comportamiento de *expand_dims* en NumPy.

Este método debe:

- Insertar una dimensión de tamaño 1 en la posición indicada.
- Recalcular correctamente la forma lógica del tensor.
- No copiar los datos almacenados.
- Garantizar que el número de dimensiones no exceda tres.

7 Concatenación

El método estático `concat` permite unir múltiples tensores a lo largo de una dimensión específica, creando un nuevo tensor con memoria propia.

```
Tensor A = Tensor::ones({2, 3});  
Tensor B = Tensor::zeros({2, 3});  
Tensor C = Tensor::concat({A, B}, 0);
```

Uso esperado:

- Construir tensores más grandes a partir de otros más pequeños.

Este método debe:

- Validar compatibilidad dimensional.
- Reservar nueva memoria dinámica.
- Copiar los datos de forma controlada.
- Devolver el tensor resultante usando `move`.

8 Funciones amigas permitidas

Se permite la implementación de las siguientes funciones amigas para realizar operaciones algebraicas que no pertenecen exclusivamente a un único objeto `Tensor`.

```
friend Tensor dot(const Tensor& a, const Tensor& b);  
friend Tensor matmul(const Tensor& a, const Tensor& b);
```

Uso esperado:

- **dot**: calcular el producto punto entre dos tensores de igual dimensión.
- **matmul**: realizar la multiplicación matricial entre tensores bidimensionales compatibles.

Estas funciones:

- Deben validar la compatibilidad dimensional.
- Pueden acceder directamente a la memoria interna del tensor.
- Deben devolver un nuevo tensor utilizando **move**.

9 Red Neuronal

Una red neuronal esta compuesta de transformaciones lineales seguidas de funciones no-lineales. Utilizar la clase **Tensor** para construir la siguiente red neuronal:

1. Crear un tensor de entrada de dimensiones $1000 \times 20 \times 20$.
2. Transformarlo a 1000×400 usando **view**.
3. Multiplicarlo por una matriz 400×100 .
4. Sumar una matriz 1×100 .
5. Aplicar la función ReLU.
6. Multiplicar por una matriz 100×10 .
7. Sumar una matriz 1×10 .
8. Aplicar la función Sigmoid.

La Tabla 1 resume el flujo completo de datos de la aplicación real solicitada, incluyendo las transformaciones de dimensiones y las operaciones realizadas en cada etapa. **Imprimir el tamaño** del tensor en cada etapa (**size**).

10 Entrega

La entrega deberá realizarse mediante un **enlace a un repositorio en GitHub**, el cual deberá contener:

- Código fuente completo (**.cpp** y si es necesario **.h**)
- Un archivo **README.md** con instrucciones de compilación y ejecución
- Comentarios claros y justificados

Paso	Operación	Dimensión resultante
1	Tensor de entrada (datos crudos)	$1000 \times 20 \times 20$
2	<code>view</code>	1000×400
3	<code>matmul</code> con pesos W_1	1000×100
4	Suma con bias b_1 (1×100)	1000×100
5	Activación <code>ReLU</code>	1000×100
6	<code>matmul</code> con pesos W_2	1000×10
7	Suma con bias b_2 (1×10)	1000×10
8	Activación <code>Sigmoid</code>	1000×10

Table 1: Flujo completo de procesamiento de la aplicación real utilizando Tensor++